Design layer

the whole layer, as one document

Contents

1 LLM Wire 4

1.1 Foundation 4
1.2 Pending updates 4
1.3 System at a glance 5
1.4 Observability and verification 6
1.4.1 Observation contract 6
1.4.2 Validation profiles 6
1.4.3 Offline qualification 7
1.4.4 Local HTTP measurement 7
1.4.5 Provider recording 7
1.4.6 Testing and extension ports 8
1.4.7 Oracle evidence 8
1.5 Extension boundaries 8
1.6 End-to-end walkthrough 9

2 Classification 12

2.1 Foundation 12
2.2 Pending updates 12
2.3 Classification 12
2.3.1 Question and candidate model 13
2.3.2 Live admission and execution 14
2.3.3 Durable receipt reconstruction 15

3 Generation 18

3.1 Foundation 18
3.2 Pending updates 19
3.3 Generation admission 19
3.3.1 Values and refinements 19
3.3.2 Configuration and preparation 20
3.3.3 Schema-bearing boundaries 21
3.4 Caller-owned conversation 22
3.5 Provider interpretation 23
3.5.1 Shared protocol contract 23
3.5.2 Built-in protocol families 25
3.6 Execution ownership 26
3.6.1 Borrowed HTTP client 26
3.6.2 State and delivery 26
3.6.3 Effect timing and deadlines 28
3.6.4 Framing and capacity 29
3.7 Failure and retry evidence 30

4 Transcription 33

4.1 Foundation 33
4.2 Transcription 33

Glossary 35

LLM Wire

1 LLM Wire

1.1 Foundation

Independent provider interactions compose with a caller-owned application.

GOAL

Return typed bounded provider interactions

Direct callers and agent runtimes prepare and execute one provider interaction (generation, classification or transcription) without acquiring a tool loop, workflow checkpoint or persistence runtime.

NO-GOAL

Own conversations or application effects

The caller owns history, tool execution, authorization, approvals, retries, round budgets, durable recovery and duplicate-effect protection.

NO-GOAL

Duplicate the generic HTTP runtime

HTTP Gun owns connection and body lifetime, TLS, destination admission, pooling, byte transport and HTTP recording and playback.

INVARIANT

Unknown evidence remains unknown

Absent usage and optional classification confidence remain None. Local cancellation proves neither provider rollback nor cancellation acknowledgement.

mechanism

PRINCIPLE

State each lifecycle's own contract

A protocol with a different lifetime states its own limits, ownership and failure evidence instead of inheriting another family’s guarantees by name.

1.2 Pending updates

verify **Establish upstream allocation guarantees** 2026-10-06

Package byte and queue bounds do not prove a strict pre-allocation bound for complete HTTP header blocks in Gun and Cowlib, native JSON allocation or arbitrary custom reducer state. HTTP parser hardening requires dependency evidence or a transport decision; no second parser and no universal memory guarantee is introduced.

ADR-0002

verify **Broaden provider and runtime conformance evidence** 2026-10-06

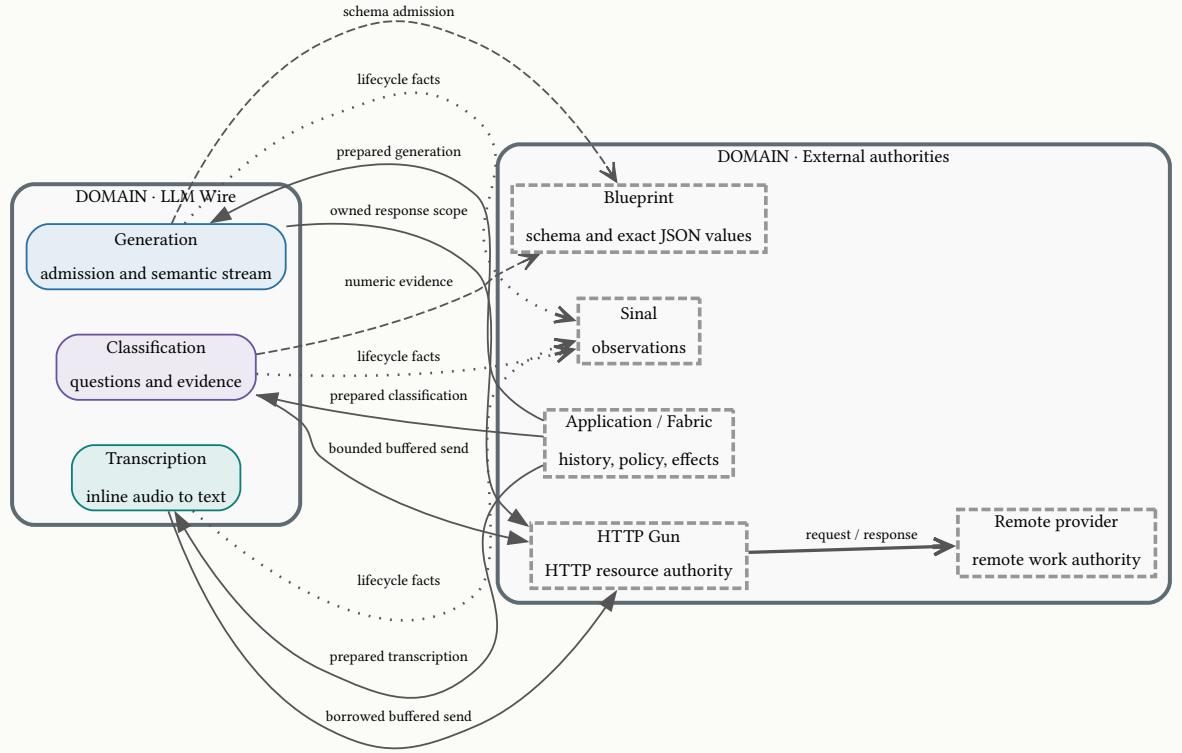
Selected local protocol, OTP, TLS and H2 cases and recorded provider cases do not establish exhaustive provider parity, every offered model, other runtime combinations or long-running soak. A new classification wire needs its own protocol and replay evidence.

ADR-0005

1.3 System at a glance

Generation, classification and transcription keep distinct models within one library.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L1 · BOUNDARY Authorities around one provider interaction



LEGEND



The application supplies its HTTP client to each family. No runtime dependency on Fabric, Saga, Relay or Grind exists.

- Generation owns request admission, configured provider encoding, semantic SSE reduction, streaming delivery and normalized outcomes. Classification owns authored questions, candidate validation and pure durable evidence decoding. Transcription owns inline audio admission and terminal text decoding through the same caller-owned HTTP authority.
- The whole model follows: request values are prepared without execution; generation creates a `Stream owner` terminal response values return to the caller. A `Classification receipt` reconstructs classification values without restarting an operation.
- The package has no published release. Present behavior lives in this design layer and the reasoning behind each change lives in the ADRs (`ADR-0016`).
- Gleam on Erlang/OTP is the execution platform. The JavaScript target is unsupported. The dependency boundaries and separation from agent state follow `ADR-0001` and `ADR-0002`.

Context	Owned model	Units
`generation`	Config, Request(o), Prepared(o), Stream(o), Outcome(o), tool declaration, assistant turn, Failure	Generation admission, caller-owned conversation, provider interpretation, execution ownership, failure and retry evidence
`classification`	Wire, live Config, Batch(a), candidates, Outcome(a), receipt	Question and candidate model, live admission and execution, durable receipt reconstruction

Context	Owned model	Units
transcription	Audio, Settings, Config, Prepared, nonempty text	Inline admission, inline protocol, borrowed buffered exchange
Root	Observation, semantic replies, HTTP exchanges, cassettes	Observability and verification, extension boundaries, end-to-end walkthrough

- Classification and transcription use the Failure, Send evidence and observation vocabulary that generation owns, and conform to it.

1.4 Observability and verification

Tests exercise real reducer paths while observations report bounded lifecycle facts.

1.4.1 Observation contract

LLM observation

responsibility

Report execution facts without transferring authority to observers.

interface

telemetry.event() is Sinal's llm_wire/observation event; Metadata holds call, correlation, stage, provider and outcome.

interactions

The execution copies correlation from the supplied HTTP Gun view so HTTP and LLM events join once.

invariants

A retry gets a fresh call id. Content, credentials, arguments and status bodies are absent from telemetry.

failure

Observation does not replace terminal state or authorize retry. Synchronous observer runtime behavior belongs to Sinal and the embedding application.

- Stages are Started, RequestSent, FirstProgress, Terminal, Cancelled, Deadline and Cleanup. RequestSent marks arrival of the response head with http_response_started, not an exact network submission timestamp. Classification's buffered path emits that stage after send returns.
- Generation emits semantic owner facts; classification and transcription emit Started, buffered response, Terminal and Cleanup. prepare emits nothing. End-to-end latency measurement surrounds execution rather than subtracting a fabricated send timestamp.

1.4.2 Validation profiles

- Required checks cover formatting, compilation and unit/integration behavior. Shared pinned Snips synchronizes README excerpts from compiled examples, with local behavior exercised by consumer tests. Other documentation consistency, source boundaries and design intent are reviewed manually; bespoke README equality checkers remain removed; rendered-layer integrity uses the imported checker; checker self-tests and receipt acceptance are not required. ADR-0018

Check	Fast	Full	Establishes
Shared formatting and design integrity	separate CI commands	separate CI commands	`nix flake check` and imported `design-gate-check`
Gleam build with warnings as errors	yes	clean build	Public types and compiler checks
Authored Erlang bridges with warnings as errors	yes	yes	Independent native FFI compilation
Gleam and classifier Python tests	yes	yes	Named semantic, local H1, TLS and server cases
External positive and negative consumers	no	yes	Public adoption and intended opacity errors
Usage examples in the public consumer	no	build and run	Documented usage compiles and runs against the current public API (ADR-0017)

Check	Fast	Full	Establishes
Local nghttpd H2 and concurrency harness	no	yes	Finite local trust, isolation and resource observations
Git diff whitespace	yes	yes	Patch whitespace

- Gates run in the pinned Nix shell against the immutable dependency revisions in the manifests. Formatting is checked on a copied Nix tree, so a gate never repairs the working tree. Frozen request fixtures, cassettes, oracle provenance, retained evidence and generated output are excluded from automatic formatting. A passing gate log is evidence for its recorded revision and runtime only.
- Gates use local and offline inputs. Static checks never execute the live recorder or read its credential source. Compiler flags reject warnings; runtime fault reports remain runtime evidence.
- CI runs the full gate on every push, pull request, manual run and weekly schedule. A final job rejects failed, cancelled and skipped mandatory jobs. The weekly run records finite HTTP observations without a latency ceiling. Sibling dependency revisions come from one pinned list, are checked out without persisted credentials, and fork pull requests run the same checks.

1.4.3 Offline qualification

- The Public consumer is also the usage documentation: one compiled, offline program per topic (defaults and limits, conversation and persistence, failures and retry advice, testing helpers, observation and client ownership) beside the flows below, with no prose usage guide to drift (ADR-0017). It exercises text, native structured output, tool rounds, streaming, retry advice, a custom provider, client supervision, early close, concurrency and script and cassette playback through public imports only. Its classification consumer proves an independent array envelope, native labels, heterogeneous questions, custom authentication and limits, absent measurements and receipts. Its transcription consumer proves common preparation, custom endpoint, model, language and mode, native result records and status failures. Negative compiler fixtures prove that opaque values stay opaque, using a positive compiled control so a missing dependency does not impersonate an intended type error.
- Reusable-settings checks exercise each provider, source-compatible custom provider fields, native output and replay, setter precedence and typed preparation refusal before extension callbacks. Deadline checks cover refusal before submission, both deadline orders, an infinite whole-call timeout, replacement of a relative default, retained failure evidence, closure of timed-out sockets and zero body and queue counters before the borrowed client is reused. Retry checks cover excessive provider delay, cumulative waiting, attempt exhaustion, affordable retry and invalid policy, and a loopback server proves that active-call time consumes the same deadline.
- Unit and reducer suites cover strict schema, output and tool admission, routing, fragmentation, usage, malformed events and semantic bounds. Owner and HTTP suites cover copied handles, conflicts, timed-read races, caller and owner death, pre-header close, absolute deadlines, slow readers and terminal-before-EOF. Local H1, TLS and H2 checks cover trust, reuse and sibling isolation under finite workloads.
- Production has no handwritten Erlang FFI; test bridges provide TCP, clocks and resource measurements. These are verification mechanisms rather than proof of universal conformance.

1.4.4 Local HTTP measurement

- A harness measures finite simultaneous demand (1, 10, 100 and 1,000 callers on one verified H2 connection to a local nghttpd) and the isolation of a cancelled stream from a sibling that collects its full response. The workload, method, recorded environment, dated results and limits are in the benchmarks reference.
- Acceptance checks semantic results, one H2 connection and cancellation isolation; timing has no speed ceiling. A run writes a fresh receipt directory recording raw consumer output and a bounded server-log prefix, and an existing directory is refused. The measurements are finite local observations and establish no provider throughput, soak result or universal memory bound (ADR-0015).

1.4.5 Provider recording

- Sanitized named OpenAI, Gemini and TypeSafe cassettes replay offline and credential-free. The recorder is opt-in, spends provider tokens and is never part of a gate; it reads provider credentials only from its documented environment variables, and credential values never enter logs, source, telemetry or documentation.
- Publication and redaction results are inspected separately from successful generation. HTTP Gun removes its documented credential metadata, while prompt bodies, queries and unlisted headers can retain sensitive content and remain the recorder owner's responsibility. A cassette proves the named request and response at its recorded revision and model, not every offered model (ADR-0013).

BEHAVIOR

A replay never reaches a provider

Given A scripted or recorded exchange set serves a prepared request.

When A request is missing, corrupt, incompatible, mismatched or beyond the exhausted exchanges.

Then The call fails without falling back to a network provider.

Then A mismatch leaves the expected exchange unconsumed.

level: boundary area: Playback

1.4.6 Testing and extension ports

Semantic test replies

responsibility

Build protocol evidence without a live provider.

interface

Opaque Reply and ScriptedCall builders describe text, tools, usage, refusal, output limit, content filter, status failures, interruption and in-band error; exchange lowers one Prepared(o) into HTTP Gun's Exchange. Classification reply builders target Prepared(a).

interactions

Scripts, cassettes and local HTTP serve the same prepared and provider runtime path. Custom providers can supply raw event chunks.

invariants

Lowering an already wire-specific reply preserves it; interrupted replies cannot synthesize success. Misuse such as a model-refusal fixture for Anthropic or Google fails explicitly.

failure

Missing, mismatched or exhausted playback stays offline and leaves a mismatched expected exchange unconsumed. HTTP Gun owns capture and publication failures separately.

- A Semantic test reply lowers into the selected provider's own wire. Failure replies (rate limited, overloaded, status, interrupted, in-band stream error, content filtered, prompt blocked and invalid output) write the error status and body of a built-in provider, raw event chunks serve a custom wire, and a retry-after value attaches to an exchange. Replies also render as an HTTP response for a fake server, and a Failure value builds directly for code that consumes one.
- High-level final-answer substitution alone is not provider-reduction evidence. HTTP Gun owns scripts, binary cassettes, strict playback, recording and publication.

1.4.7 Oracle evidence

- ReqLLM v1.24.0 is a pinned behavioral comparator, neither a dependency nor a parity commitment, and no upstream suite runs wholesale. The oracle references hold every pin, the selected cases, the exclusions and the run command. Oracle evidence pairs each selected upstream law with retained local assertions and states each difference or limit (ADR-0005).
- Exact request fixtures, malformed and fragmented SSE, offline cassettes and retained HTTP adoption receipts stay as preserved inputs. A newer model or extension needs its own named wire and ownership evidence.

1.5 Extension boundaries

Supported scope is explicit; unbuilt capabilities keep their stated owner.

Capability	Current contract	Owner of an extension
Provider-native options	Account and version fields and common sampling, token and cache settings	A complete custom adapter or a future typed provider option
Content and reasoning	Input images, semantic text, refusal and reasoning deltas, a text terminal	A future ordered-block contract that keeps protocol meaning
Tool modes and hosted tools	Application tool batch admission; hosted activity observed conservatively	A future policy with distinct provider-hosted outcomes
Model metadata	An explicit model string that the provider may reject	An injected capability description with separately owned freshness

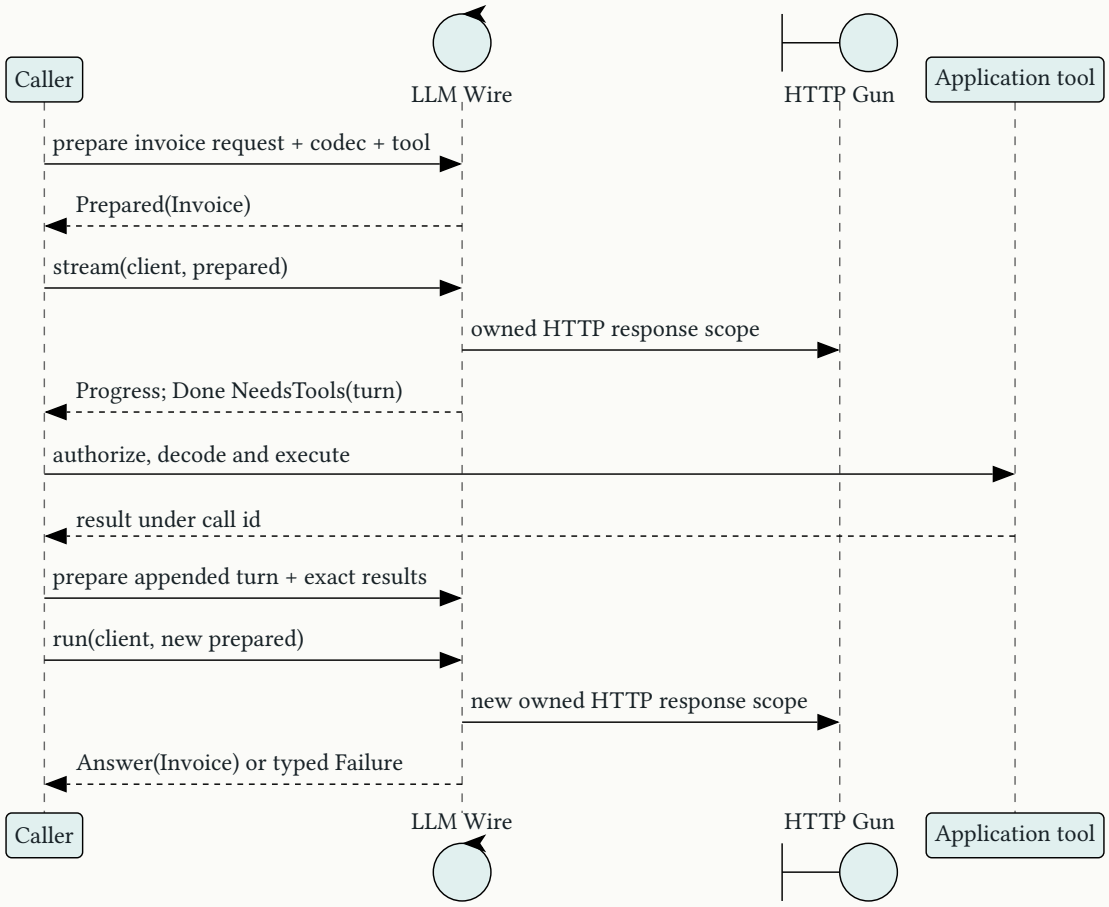
Capability	Current contract	Owner of an extension
Interaction lifecycle	One HTTP and SSE generation, buffered classification or inline transcription; local close only	A separate contract per new lifecycle
Durability	Message and replay codecs and pure classification receipts	The application: versioning and recovery, with no wire checkpoint or one-time handle

- Extensions keep the shared admission, capacity, correlation and cleanup boundary where it applies. A new protocol with a different lifetime states its limits instead of inheriting SSE guarantees by name. Pending rulings in the generation context cover the unspecified provider-native and lifecycle scope (ADR-0006).

1.6 End-to-end walkthrough

A structured extraction can ask for tools while the application retains its policy and native type.

SEQUENCE Caller-owned typed extraction and one tool round



The output type stays Invoice on the caller's request. History, approvals and execution remain visible application choices.

- The caller starts or supervises one HTTP Gun client, chooses provider configuration, declares get_weather from its native codec and creates Request(Invoice) with with_output. Preparation can reject the exact provider schema before HTTP.
- stream creates one owner and HTTP worker. Argument deltas remain progress; NeedsTools returns only after batch admission. The caller checks its registry/authorization/round budget, decodes arguments, performs the tool effect and preserves the complete assistant turn.
- Appending that turn and exact results retains the output codec and provider replay content. Re-preparing rechecks history and schemas. A final response is validated against the original Invoice contract before native decoding; unusable text remains a typed InvalidOutput failure with evidence.
- If a call fails, the caller considers advise together with send/progress and its remaining operation budget. A provider delay exceeding that budget ends the caller's attempt rather than causing an earlier retry. Closing a stream releases local resources without claiming provider rollback.

- For a durable non-generative decision, the caller instead fixes a classification wire/questions per operation version, obtains fresh live Config and runs one buffered request. It stores a receipt under its own enclosing-record bound; reopening decodes evidence through the pure fixed wire without credentials.

Classification

2 Classification

2.1 Foundation

A classification returns native answers whose evidence reconstructs without live settings.

GOAL

Return native classification answers

Typed questions yield application-native values, full distributions and optional confidence and usage from one bounded buffered request, independent of any one provider's envelope.

GOAL

Reconstruct decisions from stored evidence

A durable operation rebuilds a native outcome from its saved receipt through a pure wire, without credentials or a provider call.

NO-GOAL

Own operation storage or approval

The caller owns operation versions, the enclosing record's bound, approval and recovery. Classification owns no checkpoint or retry runtime.

INVARIANT

Malformed evidence never becomes a native outcome

Shared admission rejects missing, duplicate or unknown question ids, wrong answer kinds, incomplete distributions and inconsistent native evidence before constructing an outcome.

mechanism

INVARIANT

Wires and receipts hold no live settings

A pure wire and its receipt codec capture encoding, decoding and fixed bounds only. Credentials, endpoint overrides and live byte policy belong to live configuration.

mechanism

PRINCIPLE

Separate pure protocol from live operation

A wire is fixed for an operation version while fresh live configuration is obtained after approval or restart. Absent measurements stay absent.

composition

2.2 Pending updates

verify **Check carried-over classification bounds against code** 2026-10-17

The receipt bounds and the admission limits came from retired guides. The classification consumer exercises live admission and durable reconstruction. The numeric bounds and the evidence-failure cases are not yet checked against code.

ADR-0017

2.3 Classification

Typed questions use pure protocol adapters and shared evidence validation.

2.3.1 Question and candidate model

Question batch value-object immutable classification owned by classification questions

The typed classification definition owned by the application.

ATTRIBUTES

authored **Definitions** · unique question ids and QuestionView values
YesProbability has instructions and optional yes/no criteria; Choice has labeled alternatives; Score has ordered rubric levels. Native alternative values stay private to the batch.

derived **Native decoder** · candidate admission to a
ask names one question; combine creates a heterogeneous tuple answer. Source constructors panic on definition bugs; runtime check constructors return InvalidDefinition.

Classification wire value-object immutable classification owned by wire author

A protocol fixed independently of live operation settings.

ATTRIBUTES

authored **Projection** · provider, endpoint, pure encoder/decoder
Encoder consumes model/state and typed question views; decoder returns resolved model, candidates and optional usage. The wire must not capture credentials.

authored **Receipt bounds** · positive request and response bytes
Each defaults to 1 MiB and governs stable retained evidence.

Classification outcome value-object immutable classification owned by classification runtime

A validated native answer plus its reproducible evidence.

ATTRIBUTES

derived **Native result** · a and optional Usage
Choice keeps selected value/label, full native probabilities and optional confidence. Score keeps weighted position, ordered probabilities, rubric and optional confidence. Noul keeps yes probability.

observed **Evidence** · requested/resolved model, state, request JSON and response JSON
This material supports pure reconstruction; it contains no live authentication or HTTP process.

- Question instructions/state content is text, object or array validated under JSON structural/numeric limits; choice descriptions may explicitly be null. Ids/labels are nonempty, trimmed only for the emptiness check. Batches contain at most 256 unique questions, choices 2 through 255 alternatives, scores 2 through 10 levels.
- The candidate sum is Yes(yes), Selected(label, probabilities, confidence) or Rated(position, indexed probabilities, levels, confidence). Exact question ids/kinds and exact distribution keys are checked, including duplicates. Probabilities are finite in 0 through 1 and sum to one within 0.00001.
- A selected choice must be a known label and maximal within 0.00001. Returned probabilities are reordered to authored alternative order and retain each application value even when several labels map to the same value. A score's canonical rubric must match; its position is in 0 through levels minus one and matches the weighted distribution within 0.00001 times level count.
- Present confidence must be within 0 through 1; absent Concentration evidence remains None. Missing usage is None, and reported counts must be nonnegative. TypeSafe's protocol still requires its mandatory confidence/usage fields; shared optionality does not make malformed mandatory evidence absent.
- Shared parsing uses the already admitted byte allowance while retaining Blueprint's depth, value-count and numeric-token limits. TypeSafe validates precise decimal bounds before Float conversion and rejects positive underflow to zero. Custom wires own their corresponding precise-number and mandatory-field checks. ADR-0008 records this extension boundary.

BEHAVIOR

Malformed candidates cannot bypass shared admission

Given A wire returns missing, duplicate or unknown ids, wrong kinds, incomplete probabilities or inconsistent native evidence.

When Classification decodes its candidate answer.

Then It rejects the answer as a typed validation error instead of constructing a native outcome.

level: boundary area: Classification evidence

BEHAVIOR

Optional measurements preserve absence

Given A wire permits absent confidence or usage and returns complete valid answer evidence.

When Classification admits the response.

Then The native outcome preserves missing measurements as None.

Then Malformed evidence required by the wire remains a failure.

level: interface area: Classification evidence

2.3.2 Live admission and execution

Classification execution

responsibility

Prepare one typed decision request and execute one bounded buffered HTTP exchange.

interface

typesafe or wire produces Wire; config(reveal) produces live Config; prepare(wire, config, request) returns Prepared(a); run(client, prepared) returns Outcome(a) or Failure.

interactions

The caller supplies state, questions, HTTP client and fresh live settings. TypeSafe System One is a built-in wire; other wires can use unrelated envelopes.

invariants

Wire/receipt decoding never acquires live auth; preparation checks live/receipt compatibility before revealing credentials. Live byte allowances cannot exceed fixed receipt bounds.

failure

PrepareError names setting/unsupported question/content errors; run returns the common typed Failure with wire provider identity and conservative send evidence.

Boundary	Effect	Time and capacity
prepare	pure wire encode and bounded JSON validation; default bearer key validation after pure admission	positive live limits at most fixed receipt limits; endpoint/model/timeout valid
custom headers	replace bearer auth; unused bearer closure is not invoked	header reveal occurs when building the HTTP request for execution
run	fresh call id/correlation; HTTP send buffers response under response limit	600 s default request timeout; Infinity explicit; HTTP client idle/connect/pool policies remain
response	UTF-8 check, status, bounded JSON, wire decode, candidate admission	non-200 status has empty retained body and optional Retry-After; response body limit still applies
success	Outcome includes native answer and original request/response/state	no tool loop, SSE stream or generation first-progress/idle timers

- Live Config holds auth, optional endpoint override, timeout and live request/response bytes, each defaulting to 1 MiB. The pure wire is fixed separately; a Fabric operation/version owns that choice while providing fresh live configuration after approval/recovery. No Fabric type is needed in LLM Wire.
- Default bearer reveal validates visible ASCII/nonempty after pure admission and is invoked again at HTTP request construction; auth freshness is a caller callback property. with_headers replaces it without revealing or validating the unused key. HTTP Gun remains responsible for admitting the resulting request.
- Classification endpoint validation admits HTTPS or loopback HTTP and requires a nonempty path with no userinfo/query/fragment/controls. Plaintext execution uses the same destination setup described under Execution ownership. It replaces request timeout and body limit; it does not lift HTTP idle as generation does.
- The classification timer is HTTP Gun's buffered send budget. Synchronous auth callbacks before send and JSON/native decoding after send have no independent preemption; it is not a wall-clock guarantee over arbitrary callbacks. A successful HTTP response with unusable evidence has Completed send evidence, partial_output False and no failure usage retained by current failure_for.

Classification bound	Default	Owner of the change
Whole request timeout	600 seconds, or an explicit Infinity	Live config
Request JSON and response JSON	1 MiB each	Live config, at most the wire's receipt bounds
Receipt request and response JSON	1 MiB each	The wire
Questions per batch	256	Fixed
Choice alternatives and score levels	2 through 255 and 2 through 10	Fixed

- Raising a byte allowance does not raise Blueprint's structural limits (nesting 64, 262,144 values and bounded number tokens). The HTTP client's own request, buffering and collection limits are independent and can impose a smaller allowance. A call neither retries nor follows redirects, and keys enter only through a reveal closure.

BEHAVIOR

Receipt limits are checked before credentials

Given A live byte allowance is nonpositive or exceeds the wire's fixed receipt allowance.

When The caller prepares classification.

Then Preparation returns InvalidSetting for that limit.

Then No credential closure is invoked and no provider request occurs.

level: interface area: Classification preparation

2.3.3 Durable receipt reconstruction

Classification receipt codec

responsibility

Reconstruct a native answer solely from saved protocol evidence.

interface

receipt_codec(wire, questions) is a Blueprint Codec(Outcome(a)) with no schema; supports the llm.classification.receipt.v1 and fabric.typesafe.receipt.v1 tags.

interactions

The embedding store versions its operation/wire and separately bounds the enclosing record, including escaping and state.

invariants

The reconstructed request equals saved request canonically and response candidates satisfy the same authored questions. Encoding rejects forged native outcomes.

failure

Wrong format, oversized evidence, changed questions or inconsistent response/native answer fails codec encoding/decoding; no credentials or provider call is used.

- The array tag llm.classification.receipt.v1 carries requested model, state, request JSON and response JSON. The tag fabric.typesafe.receipt.v1 contains request and response only; model and state are recovered from the bounded request. Both follow the same reconstruction.

- Restoration bounds saved request/response, re-encodes expected questions/model/state under fixed wire bounds, parses and canonicalizes object member order, compares saved/expected requests, decodes/admit response and constructs native Outcome. Encoding additionally compares the reconstructed outcome with the supplied outcome.
- Wire identity/projections and receipt bounds are compatibility facts of the embedding operation version. Live endpoint, key and smaller live limits may change without making old receipts unreadable. Raising evidence allowance requires explicitly raising fixed receipt and live allowances; the outer storage record still needs its own bound. ADR-0008 records why these policies stay distinct.

BEHAVIOR

Receipt decoding is independent of live settings

Given A saved receipt uses the fixed wire/questions and compatible bounded evidence.

When Its receipt codec decodes it.

Then It reconstructs the native answer and retained evidence without live configuration, credentials or I/O.

level: boundary area: Receipts

BEHAVIOR

Changed evidence refuses durable reconstruction

Given A saved request disagrees with deployed questions or a native outcome disagrees with its protocol evidence.

When The codec decodes the receipt or encodes that outcome.

Then It returns a typed codec failure instead of accepting the inconsistent record.

level: boundary area: Receipts

CONTEXT

Generation

3 Generation

3.1 Foundation

One generation is one prepared request, one owned semantic stream and one typed outcome.

GOAL

Return typed bounded generations

Direct callers and agent runtimes prepare, execute and stream one provider interaction without acquiring a tool loop or persistence runtime. OpenAI Responses, Anthropic Messages and Google GenerateContent are the baseline provider targets; a public custom adapter uses the same execution API.

GOAL

Preserve caller values and provider meaning

Structured generation returns the application's output type. Provider replay requirements survive in response data. Reusable generation settings configure sampling, limits and cache references without owning conversation or output types.

NO-GOAL

Own conversations or application effects

The caller owns history, tool execution, authorization, approvals, retries, round budgets, durable recovery and duplicate-effect protection. Generation owns no workflow checkpoint or background agent.

INVARIANT

Preparation opens no package-owned resource

Preparation performs local validation and encoding before transport. It starts no owner, timer or socket and emits no execution observation. Extension callbacks must respect their declared pure contract.

mechanism

INVARIANT

Incomplete arguments never become executable calls

Argument deltas are progress only. A normal tool-request terminal requires a complete admitted batch; output-limited calls remain explicitly partial.

mechanism

INVARIANT

One owner decides each stream terminal

All handles share terminal state. Terminal processing closes local transport; a later event cannot replace the outcome.

mechanism

PRINCIPLE

Keep policies with their actual authority

Adapters encode and interpret provider protocols. Shared runtimes keep admission, limits, correlation and resource cleanup. Applications decide whether another interaction is justified.

PRINCIPLE

Reject unsupported requests before effects

A valid schema outside the selected provider profile fails explicitly. Unsupported requested features must not be silently discarded.

3.2 Pending updates

ruling **Resolve stream read authority** 2026-10-06

The public Stream documentation permits only the creating process to read, while the owner accepts reads from other processes and allows one pending read. Settle which read authority the public contract promises.

ADR-0007

ruling **Specify richer provider-native generation** 2026-10-06

Sequential and parallel tool mode, tool choice, richer native options, optional injected model capabilities, provider-hosted tool outcomes, richer response blocks and typed extension payloads are unspecified. Native options cover credentials, account and version fields; adding a request field requires a complete custom adapter. A concrete advanced consumer must exist before a narrow provider option or an admitted request extension is chosen.

ADR-0006

ruling **Specify advanced interaction lifecycles** 2026-10-06

Richer audio and video, embeddings, realtime and WebSockets, provider batch and background work, interrupted-stream resumption and optional remote cancellation acknowledgement need distinct protocol, state, ownership and compatibility contracts. They are not generation guarantees.

ADR-0006

verify **Check carried-over usage claims against code** 2026-10-17

The defaults and timer composition under Effect timing and deadlines, the extension rules under Provider interpretation and the retry guidance under Failure and retry evidence came from retired guides. The compiled usage examples exercise the defaults, conversation persistence, failure kinds, offline testing, observation and client ownership. The remaining statements in those sections are not yet checked against code.

ADR-0017

3.3 Generation admission

A reusable prepared value binds one encoded request to its output decoder.

3.3.1 Values and refinements

Generation request value-object immutable generation owned by caller

The caller's explicit request and desired native output.

ATTRIBUTES

authored **Input** · model string and ordered List(Message)
Messages hold system/user text, user content parts, assistant turns, assistant content parts or tool results. Content parts are text, remote image URL or inline MIME/base64 image.

authored **Options** · optional maximum tokens, temperature, top-p and cache reference; stop list
Maximum tokens is positive; temperature is within 0 through 2; top-p within 0 through 1. Model/output name is trimmed and nonempty. Cache references are nonempty and provider-specific.

authored **Output** · String or Blueprint Codec(o)
Plain text returns String. with_output changes only the output type and retains the remaining request settings.

RELATIONSHIPS

1 : 0..n A request declares zero or more tools and contains an ordered caller-owned conversation.

Prepared generation

value-object · immutable · generation · owned by generation admission

The opaque result of admitting one configured request.

ATTRIBUTES

derived **Admitted material** · adapter, normalized request, tools, body and output contract
The encoded body and admitted schemas are fixed. Headers are captured behind an inspection-safe closure.

derived **Execution settings** · limits, timeouts, tool-check policy and Codec(o) decoder
Reusing this value creates another execution and a new call identifier. It grants no single-use or duplicate-effect protection.

RELATIONSHIPS

1 : 0..n One prepared generation can create multiple independent executions.

- Public constructor values are opaque where arbitrary construction could bypass admission. Returned records are read by label; extensible sums use a fallback arm. Generation's Config, Request(o), Prepared(o), Stream(o), tool declaration and Adapter keep private machinery out of caller type parameters. ADR-0003 records this facade.

3.3.2 Configuration and preparation

Reusable generation settings

responsibility

Retain generation controls independently of a conversation.

interface

generation.Settings contains optional maximum output tokens, temperature, top-p, stop sequences and one provider cache reference. generation.default leaves optional controls absent and stop sequences empty. with_generation replaces all controls; existing request setters update the same canonical settings.

interactions

Request retains model, messages, settings, tools and output independently. Preparation derives the existing provider.Request field-label view without changing the custom adapter contract.

invariants

Settings contain no model identity, messages, tools, credentials, output codec or runtime state. Changing output type or appending conversation preserves settings. Anthropic's default output bound remains an encoding choice.

failure

Construction does not validate provider compatibility. Existing prepare checks reject invalid controls before encoding or HTTP execution.

BEHAVIOR

Generation settings have replacement semantics

Given A request has generation controls.

When The caller applies a Settings value.

Then All generation controls equal that value.

Then Model, conversation, tools and output remain unchanged.

level: interface · area: Preparation

- A scalar request setter after with_generation overrides only its named control. Applying generation.default later clears all explicit controls. ADR-0011

Generation preparation

responsibility

Validate and encode one request without networking.

interface

prepare(Config, Request(o)) returns Prepared(o) or PrepareError; request_json reads the encoded body without credentials.

interactions

Uses adapter validation, endpoint admission, tools, schema projection and the caller's output codec.

invariants

Selected provider, admitted contracts and native decoder remain together. Invalid local values never reach HTTP.

failure

InvalidSetting, InvalidRequest, UnsupportedSchema, ToolResultMismatch or RequestTooLarge name the refused boundary. Diagnostic strings are not classification keys.

- Each built-in provider is selected by its constructor and configuration function (OpenAI, Anthropic, Google), and an endpoint override selects a compatible server.
- Built-in settings hold their keys behind reveal closures. OpenAI adds organization/project; Anthropic adds API version; Google adds API version. Common settings own limits, three time bounds and tool-call checking. All bounds must be positive unless an explicit timeout Infinity is selected.
- Preparation validates timeouts, adapter settings, positive limits, trimmed model, historical turns, unique tool names and projected tool/output schemas. It then validates endpoint and common/provider options, calls the encoder, reveals and validates provider headers, validates the joined path and bounds the encoded body.
- Generation endpoint admission requires HTTP(S), a host, an admitted port and a path without userinfo, query, fragment or controls. Encoded paths cannot inject a new authority or controls. Header names/values and reserved common headers are checked; credentials remain inspection-safe but the application must govern logging of prompt bodies.
- OpenAI refuses stop sequences; Anthropic and Google admit finite lists, with Google's maximum five. OpenAI admits remote image URLs; Anthropic and Google require inline image content. Cache references are OpenAI prompt_cache_key or Google cachedContent; other providers reject them.
- Custom projection/encoding/header callbacks run synchronously during preparation. Their declared purity excludes networking and uncontrolled effects; the type system cannot enforce that promise. Generation header values are captured during prepare, so changing credentials requires preparing again.

BEHAVIOR

Invalid local input fails before networking

Given A generation request has an invalid configuration, option, schema or tool-result association.

When The caller prepares the request.

Then Preparation returns the relevant typed preparation error.

Then No generation execution starts.

level: interface area: Preparation

3.3.3 Schema-bearing boundaries

Schema location	Admitted shapes	Refused shapes
Built-in tool parameters	string/enum, integer/range, number, bool, any; recursive list, nullable, object	pair, union, number range, unknown kind
OpenAI/Anthropic output	record root; required closed nested objects, lists, nullable, enums, integers/ranges and nested tagged union	non-record root, optional property, pair, number range, any, unknown kind
Google output	record root; also optional fields, pair, number range and any below root	non-record root and unknown kind
Custom adapter	its explicit projection callbacks; Blueprint's canonical schema by default	whatever its projection returns as unsupported

- Nested Blueprint unions become anyOf of closed variant objects; each tag is a one-element string enum. The original codec still decodes the response, so this projection cannot bless a different native value. Google's field is

generationConfig.responseJsonSchema; tool declarations use parametersJsonSchema. Anthropic documents that enum and const capitalization is not guaranteed, so the codec's decoding is the final check on a tag. ADR-0004 records protocol-specific evidence.

- tool.new is for authored declarations and panics with the tool name when its name/schema is invalid. Runtime from_contract/from_json_schema returns ToolError. Tool names match ASCII letters, digits, underscore or hyphen, with length 1 through 64. Schema-only MCP input needs no Relay type or runtime dependency.
- Schema-only admission inherits Blueprint's supported profile and loader failures. It is not general JSON Schema support; unsupported external MCP schemas remain explicitly refused. Native tool arguments and final output are parsed within byte/depth bounds, validated against the original contract, then decoded. Schema-invalid values never enter codec mapping callbacks.

BEHAVIOR

Unusable structured output retains its evidence

Given An admitted structured request receives final text that is malformed JSON, schema-invalid or rejected by its codec.

When The completed output is decoded for the caller.

Then The result is InvalidOutput with the original text and typed ValueFailure.

Then Send evidence is Completed and reported usage remains available.

level: interface · area: Output validation

3.4 Caller-owned conversation

A tool terminal returns response data for the caller's next ordinary request.

Assistant turn

value-object · immutable · conversation · owned by message model

One response value preserved by the caller.

ATTRIBUTES

observed **Response** · provider option, text, calls, response-id option, provider-data option
Application-written turns may omit provider. A provider-produced turn names its provider and retains required replay data.

RELATIONSHIPS

1 : 0..n One assistant turn carries an ordered batch of tool calls; each call has one round-local application correlation id.

Tool call

value-object · immutable · conversation · owned by provider response

A provider's request for an application tool result.

ATTRIBUTES

observed **Identity** · id and name strings
The call id differs from provider decoder routing keys. Ids may repeat in different assistant turns; names are constrained by the common tool-name grammar.

observed **Payload** · arguments JSON and optional provider id/state
Provider id/state is replay metadata. Argument text remains exact response evidence until native decoding.

Conversation validation

responsibility

Validate each historical assistant batch and following results.

interface

append retains a request's options and output codec; message codecs store response data; prepare validates the resulting transcript.

interactions

The caller executes tools and appends Assistant(turn) plus one ToolResult per call. The provider encoder interprets message-local replay data.

invariants

Every call has exactly one immediately following result; result order is normalized to call order. Historical calls need not belong to the current catalog.

failure

Missing, duplicate, unknown or unassociated result ids fail before transport; turns from another provider and inconsistent replay data are refused.

- A tool-request terminal must contain a nonempty batch with unique ids, finite metadata/arguments and complete provider blocks. Unknown tools and invalid arguments fail by default; ReportInvalidToolCalls retains all calls with Tool call issue values. Duplicate identities and capacity violations always fail.
- A returned call does not grant application registry membership, authorization or execution permission. Reporting a bad call makes it answerable as a tool error message; it does not make it executable. The caller owns round limits, approvals and retry policy.
- Google signed parts are retained in each turn's Replay data. Encoding checks that raw parts agree with normalized text/calls and preserves signed content/order; unsigned parts use canonical argument repair. Custom interpretation belongs to the configured adapter.
- message.to_json/decoder uses llm_wire.message.v1; turn_to_json/turn_decoder uses llm_wire.turn.v1. A missing format tag is read as version 1; a different tag is rejected. turn_replay_to_json stores provider/id/data alone; its decoder takes separately stored text/calls and ignores an obsolete issues member in Fabric records.
- Message decoders restore data shape rather than admission or trust. The storage reader bounds its enclosing record; preparing a restored conversation revalidates it. Neither saved turns nor prepared values provide one-time consumption. ADR-0001 records the replacement of wire-owned continuation.

BEHAVIOR

A follow-up requires exact local result coverage

Given An assistant message contains tool calls and the caller supplies a follow-up request.

When The follow-up is prepared.

Then Missing, duplicate or unknown result ids are refused before transport.

Then An admitted request places results in that assistant turn's call order.

level: boundary area: Conversation

BEHAVIOR

Reporting policy retains invalid calls

Given A complete bounded tool batch contains an undeclared tool or arguments rejected by its declaration, and reporting is selected.

When The owner admits the completed tool-request terminal.

Then NeedsTools contains every call and the corresponding typed issues.

level: interface area: Tool admission

3.5 Provider interpretation

Each adapter translates wire events without acquiring execution policy.

3.5.1 Shared protocol contract

Provider adapter

responsibility

Encode admitted requests and reduce provider events into semantic progress and terminals.

interface

provider.new accepts identity, endpoint, encoder and reducer factory; schema projection/header callbacks are separate; reducer state remains hidden.

interactions

Shared admission supplies projected tools/output; the owner supplies finite SSE events and validates returned progress and terminals.

invariants

Callbacks cannot bypass shared tool-batch, metadata, text and queue admission. Unknown events do not become application tool calls.

failure

Encoder/projection errors refuse preparation. Reducer errors terminate execution with local cleanup; custom state size and synchronous callback termination remain adapter-author obligations.

- Progress alternatives are TextDelta, RefusalDelta, ReasoningDelta, ToolArgumentsDelta, UsageUpdate and ProviderExtension. Public extension values retain a bounded event name, not an arbitrary raw payload. A custom adapter returns text, tool_calls, output_limited, refused or failed terminal values.
- Built-in reducers track provider routing keys, ordered blocks, completion flags, usage and observed evidence. Deltas after closure, contradictory routing, duplicate starts/ids, malformed arguments or incomplete normal terminal calls fail. Multiple tool calls remain in declaration order despite interleaved completion.
- Outcome(o) separates Answer(output, text, usage), NeedsTools(turn, issues, usage), OutputLimited(partial_text, partial_calls, usage), and Refused(reason, usage). OutputLimited data is incomplete evidence and must not enter normal tool dispatch. ContentFiltered is a failure, distinct from a model's own refusal.
- Usage is optional. Anthropic cumulative output updates replace the prior total rather than being summed; Google/OpenAI report their own final or updated counts. No provider usage parity is claimed for detailed cache/reasoning billing.

Provider block state absent · open · complete · invalid

Provider block

entity · stateful · generation · owned by provider reducer

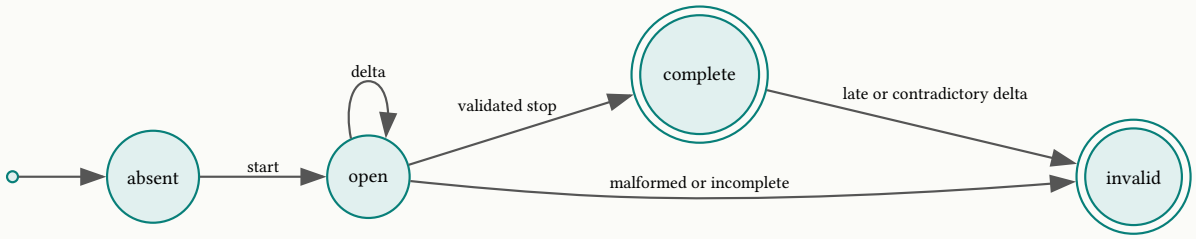
A routed text or argument buffer in one reducer.

ATTRIBUTES

derived **State** · Provider block state block-state · block-lifecycle
A block is absent until its declared start; closed content cannot accept further deltas.

observed **Routing and bytes** · provider key, application id where relevant, bounded buffer
Routing keys remain internal. Completed argument text is parsed before a normal tool terminal.

STATE MACHINE A block accepts fragments before completion



The owner converts invalid grammar into one stream failure. This common model does not replace each protocol's routing rules.

3.5.2 Built-in protocol families

Provider	Routing and encoding	Terminal and exceptional meaning
OpenAI Responses	output_index and item id route text/reasoning/function arguments; application correlation uses call_id; input/output contracts use Responses forms	completed/incomplete/failed and in-band error are distinct; max_output_tokens is output-limited; content_filter is ContentFiltered; refusal content is Refused
Anthropic Messages	content block index routes text/tool_use; tool_use.id is application id; result is user tool_result; system text is separate	message_stop requires completed blocks; max_tokens is output-limited; refusal stop is ContentFiltered; server_tool_use is tracked separately from application calls
Google GenerateContent	model path streamGenerateContent with SSE; functionCall ids are retained or locally synthesized; model/user parts carry signed replay content	STOP completes; MAX_TOKENS is partial; promptFeedback and safety finish reasons become ContentFiltered; provider errors and unknown stop reasons fail explicitly

- OpenAI unknown named events produce bounded ProviderExtension. Anthropic ping produces no progress and unknown named events produce extensions. Google reduction reads its JSON payload rather than relying on named SSE events. Usage/extension/ping activity cannot end the first-progress wait.
- Provider-hosted actions can already have effects. Anthropic server-tool observation and ambiguous tool activity strengthen internal uncertainty; these do not authorize application dispatch. A public structured provider-hosted outcome remains an explicit pending contract.
- Generation wire JSON uses standard Gleam JSON; duplicate-member rejection is not universally promised for provider envelopes. Tool/output schema values use Blueprint so exact schema numbers do not pass through a Float bridge. ADR-0004 and ADR-0005 distinguish profile support from measured provider evidence.

BEHAVIOR

A protocol failure ends the owned stream

Given A provider sends malformed or contradictory events before the stream ends.

When The interaction interprets those events.

Then The stream ends with a typed protocol failure and releases local transport.

Then It returns no normal completed tool batch from the partial arguments.

level: interface area: Provider reduction

BEHAVIOR

Content filtering preserves provider meaning

Given A built-in provider reports an admitted prompt or output content-filter stop.

When The reducer handles that filtering terminal.

Then The failure contains ContentFiltered with stage and the provider's reason, and Completed send evidence.

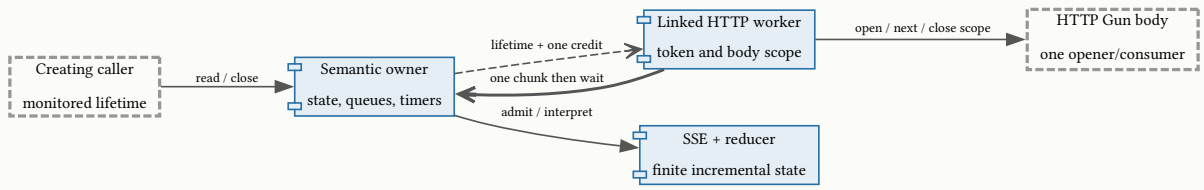
Then Retry advice says WillNotHelpUnchanged.

level: interface area: Terminal outcomes

3.6 Execution ownership

One generation owner serializes state while one linked worker consumes HTTP bytes.

VIEWPOINT RUNTIME · ALTITUDE L3 · COMPONENTS — One stream execution and its resource authorities



LEGEND



The stream handle names the owner. The worker alone opens and reads the HTTP body; cancellation covers its whole scope.

3.6.1 Borrowed HTTP client

- The application starts one HTTP Gun client at startup, passes it to every run and stream, and owns its whole lifetime; generation never starts or stops it. A client supervised under a name stays valid across restarts.
- A prepared generation can execute more than once and each execution starts a new owner and call identifier. Whether to execute again is the caller's decision.

3.6.2 State and delivery

Stream state unstarted · open · terminal-pending · ended

Stream execution aggregate · stateful · execution · owned by stream owner

One authoritative process for one started generation.

ATTRIBUTES

derived **State** · Stream state stream-state · stream-lifecycle
The terminal slot latches the first outcome. Accepted queued progress precedes that terminal unless explicit local close ends delivery.

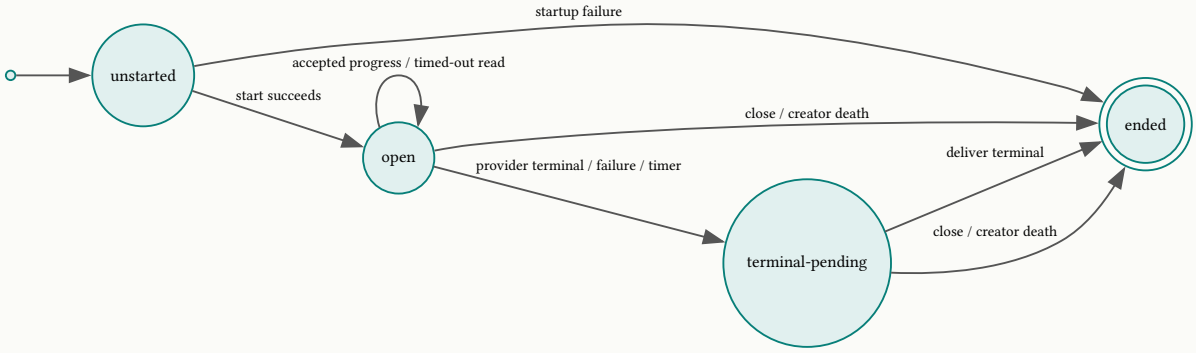
derived **Delivery** · bounded FIFO, one optional pending read and read identity
Count/byte counters track the FIFO. Read timeout arbitration decides whether cancellation or delivery won; waiting readers are monitored.

derived **Resource and evidence** · transport port, credit, timer handles, monotone evidence, last usage
Creator death cancels the call. The transport-closed flag guards normal-path local cleanup.

RELATIONSHIPS

1 : 1 Each started owner holds one linked transport worker and one private reducer.

STATE MACHINE A terminal closes transport before delivery ends



Terminal pending retains already queued progress but no live HTTP scope. There is no transition back to open; a prepared value starts a new owner.

Read and close protocol

responsibility

Serialize delivery and terminate locally when requested.

interface

stream returns before HTTP headers; next waits, next_within adds a local wait, collect discards progress; close returns Closed or AlreadyEnded.

interactions

Reads share one owner and FIFO. Close latches cancellation; pending-reader death clears only its read. Creating-caller death releases the whole execution.

invariants

One pending read; one terminal; copied handles share closure. A read timeout leaves the stream readable.

failure

ReadError is StreamEnded, ConcurrentRead, OwnerGone or TimedOut. Owner startup fails as Stopped/NotSent; loss while collecting returns conservative Stopped/MaybeSent evidence.

- The owner accepts cross-process reads and rejects another pending reader; its public comment declares creator-only reads. Lifetime ownership and read authority are distinct. The pending ruling preserves this conflict.
- Close and owner exit settle against pending delivery. A terminal already delivered wins; repeated close is idempotent even when the owner has exited. A queued provider terminal cannot be replaced by late HTTP events. Early provider completion closes transport without draining to HTTP EOF.
- Normal cleanup cancels timers, closes the transport once and clears pending monitors. Abnormal owner death terminates its linked worker; HTTP Gun observes opener death. Command cardinality does not prove exactly-once cleanup effects under arbitrary process loss.

BEHAVIOR

A read wait expires without cancelling generation

Given A stream remains active without an event during the caller's shorter read wait.

When The caller reads with next_within.

Then It receives TimedOut unless delivery wins the same race.

Then The stream remains available for a later read.

level: interface area: Read delivery

BEHAVIOR

Close is shared and idempotent

Given Multiple handles name the same stream.

When A caller closes one handle.

Then The active local interaction is cancelled or an existing end is reported.

Then Other handles cannot keep independent live executions.

level: interface area: Lifetime

BEHAVIOR

Creator death releases the interaction

Given The process that started a stream exits while it is opening or reading.

When Its exit is observed.

Then The interaction cancels and releases its local connection resource.

level: boundary area: Lifetime

3.6.3 Effect timing and deadlines

Boundary	Effect and time owner	Failure or limit
prepare	local projection/encoding and generation header reveal; no execution clock	typed pure admission error; trusted callbacks must terminate
stream	fresh call id; absolute whole-call deadline before reducer/owner setup	Stopped if owner setup fails; remaining deadline reaches HTTP and owner
owner setup	linked worker plus first-progress timer and creator monitor	startup wait is bounded by actor/worker setup; synchronous adapter factory is not preempted
HTTP opening	worker token scope; HTTP Gun admission/ connect/headers consume whole-call budget	typed HTTP failure or WholeCall deadline
SSE interpretation	first semantic progress cancels FirstToken and starts IdleGap; every framed event then resets idle	comments/bytes alone do not reset semantic idle
terminal	local cancellation/close immediately; queued delivery may follow	no remote rollback or cancellation guarantee
structured decoding	caller-side output validation after semantic terminal and HTTP cleanup	InvalidOutput; synchronous codec callback has no independent preemption

- WholeCall defaults to 600 seconds. FirstToken defaults to 180 seconds; it begins during owner setup, after reducer factory and transport initializer work, and ends at the first accepted semantic progress. IdleGap defaults to 60 seconds after that point. Infinity lifts each independently. Preparation and caller retry waiting are outside these timers.
- Generation establishes one absolute whole-call deadline before setup. The owner uses its remaining duration, and HTTP opening replaces the relative request timeout with the remaining duration at dispatch. An explicit deadline on the supplied HTTP client remains a ceiling, including when WholeCall is Infinity. Generation lifts HTTP idle; connect and pool admission bounds still apply. ADR-0010 records the distinction between a default timeout and an explicit deadline. Synchronous custom factory/reducer/output callbacks are trusted code and may delay observing elapsed time.
- For HTTPS, the supplied client's destination policy remains. For admitted plaintext, the implementation configures default destination policy with loopback/private admission and PlaintextToLoopbackOnly through with_destination. HTTP Gun combines view constraints; this setup is not a claim that a caller's refusal can always be overridden or merely narrowed. Resolved non-loopback plaintext remains refused.
- Status 200 requires text/event-stream and absent/identity encoding. Other status bodies are bounded by ErrorBodyBytes and retained as Status with Retry-After; body/stream errors preserve independent observed-byte evidence. HTTP Gun does not add retries, redirects or decompression to this path.

BEHAVIOR

An application deadline constrains generation

Given The supplied client has an explicit absolute deadline.

When Generation opens or consumes its response.

Then The earlier of the application deadline and the whole-call deadline bounds the provider interaction.

Then An expired application deadline refuses submission with NotSent evidence.

Then Expiry closes local response resources while preserving observed send, progress and usage evidence.

Then The caller retains ownership of the supplied client.

level: boundary area: Deadlines

BEHAVIOR

A generation timer ends local execution

Given An executing generation exceeds an applicable whole-call, first-progress or idle-gap timer.

When The deadline is observed.

Then The interaction ends with DeadlineExceeded naming that timer and local cleanup.

Then It does not restart the request.

level: interface area: Deadlines

3.6.4 Framing and capacity

Incremental SSE framer

responsibility

Reconstruct finite events from arbitrary binary chunks.

interface

feed consumes bytes and returns events plus remaining framing state; finish handles EOF.

interactions

The owner checks response bytes, frames events and steps its provider reducer; the worker waits for further credit.

invariants

LF/CRLF, split CRLF and split UTF-8 retain their meaning; line/event growth and per-chunk event counts are finite.

failure

Invalid UTF-8 or malformed framing fails; incomplete EOF is handled explicitly rather than manufacturing a provider terminal. LimitExceeded names the violated setting.

- The framer retains an incomplete binary line, partial event fields, data fragments and scan position. Comments are ignored; multiple data lines join with newlines; event/id/retry fields retain their typed SSE meanings. A blank line dispatches an event. A long incomplete line resumes scanning rather than rescanning all prior bytes.
- One worker sends one chunk then waits for one credit. The owner requests more only below count and byte watermarks with no outstanding credit. Events already present in a chunk may overflow and fail; pull shape does not promise that one incoming chunk cannot contain many events. The terminal has a separate slot so a full progress queue cannot erase it.
- The owner rechecks custom-provider progress text/block/extension and terminal text/arguments/metadata. Built-ins additionally bound their own block buffers. Common checks cannot prove an arbitrary custom reducer's captured state is finite. HTTP Gun's upstream body queue is a separate bound.

Limit	Default	Boundary
RequestBytes	1 MiB	encoded generation body before execution
ChunkBytes	64 KiB	incoming transport chunk
LineBytes / EventBytes	1 MiB / 1 MiB	incremental SSE line/event

Limit	Default	Boundary
ResponseBodyBytes / ErrorBodyBytes	8 MiB / 64 KiB	stream bytes / retained error body
QueueCount / QueueBytes	500 / 2 MiB	undelivered progress and framer event capacity
ActiveBlocks	64	active reducer blocks and returned call count
TextBytesPerBlock / TotalTextBytes	1 MiB / 4 MiB	semantic text and terminal validation
ArgumentBytesPerCall / TotalArgumentBytes	1 MiB / 4 MiB	tool argument buffers and admitted terminal
ProviderMetadataBytes	1 MiB	id/name/signature/raw replay aggregate per turn
ExtensionBytes	16 KiB	uninterpreted event name

BEHAVIOR

A capacity violation ends the interaction

Given A generation exceeds a configured byte or count bound.

When The bounded input or output is admitted.

Then The call fails with LimitExceeded naming the setting and measurement, or RequestTooLarge during preparation.

Then No silent truncation makes the value appear complete.

level: interface area: Capacity

3.7 Failure and retry evidence

A prospect for another attempt is independent of proof about provider work.

Failure field	Meaning	Caller obligation
error	Http, Status, Provider, Protocol, LimitExceeded, DeadlineExceeded, Cancelled, InvalidOutput, Stopped, ContentFiltered	branch on variants/closed Kind; diagnostics are for people
sent	NotSent, MaybeSent or Completed	do not infer rollback or free repeated work
partial_output	semantic progress accepted before failure	accepted progress may already be externally visible
provider / usage	origin and last reported token counts	retain unknown usage rather than zero

- error.kind classifies every error into the closed set Transport, ProviderError, ContentPolicy, UnusableResponse, OverLimit and Ended. The set gains no variants, so a caller's exhaustive match stays complete.
- Internal evidence retains NoRequestSent, RequestMayHaveReachedProvider, ResponseCompleted or EffectUnknown plus byte/progress flags. Independent observations are merged monotonically; stronger uncertainty cannot be erased by a later HTTP error. Public `Send evidence` deliberately collapses uncertain effect to MaybeSent.
- advise reads Failure alone. HTTP unavailable/network/timeout failures may help; invalid/refused/too-large/cancelled/misuse/playback failures will not help unchanged. DeadlineExceeded may help; capacity, local cancellation and filtering will not; protocol, invalid output, stopped and unknown provider codes remain Unknown.
- Statuses 408, 429, 500, 502, 503 and 504 may help; Anthropic 529 may help. Common client/account/request rejection statuses will not help unchanged. Provider error-code mappings are exact, per provider; diagnostic prose is never parsed.
- RetryDelay is `Provider delay` when Retry-After is readable, otherwise Backoff. Delay seconds and HTTP dates are supported; a past date yields zero. Prospect says neither replay-safe nor successful: the caller weighs sent/progress, hosted effects, remaining deadline, budgets and its chosen retry owner. `ADR-0003` records this distinction.
- LLM Wire performs no automatic retry. Scheduling a provider delay as a snooze may preserve a queue attempt, while ordinary queue retry consumes one under Grind's own contract. The application translates that distinction without a shared retry runtime.

BEHAVIOR

Retry advice never schedules another call

Given A caller has a failed interaction.

When It requests retry advice.

Then It receives a prospect and provider delay or caller-backoff choice.

Then No wait, new request or tool effect occurs.

level: interface area: Retry advice

BEHAVIOR

The retry consumer bounds active work and waiting

Given The caller supplies a prepared interaction, finite total duration, maximum attempts and fallback delay.

When The consumer executes its retry loop.

Then The consumer admits positive attempts and a total budget of 1 through 4,294,967,295 whole milliseconds, within the BEAM receive timeout range. Invalid budgets or negative fallback fail locally without starting a call.

Then One monotonic deadline accounts for calls and waits. Each stream read uses the remaining duration; expiry closes the owned stream and returns an application deadline error without claiming provider rollback.

Then A provider delay or fallback that cannot fit before the deadline returns the original typed provider failure without shortening the wait or starting another call.

Then The supplied client remains caller-owned. Startup, read-timeout arbitration and cleanup retain their own runtime bounds. Synchronous extension callbacks are not preempted; the budget does not promise an exact return-time ceiling. ADR-0012 records this boundary.

level: interface area: Retry consumer

Transcription

4 Transcription

4.1 Foundation

One inline audio request returns completed text while the caller keeps transport and recovery policy.

GOAL

Return completed text from inline audio

A caller turns bounded inline audio into a nonempty native String through one Google Interactions request.

NO-GOAL

Manage audio resources or recovery

Uploads, diarization, timestamps, custom vocabulary, streaming audio, persistence, duplicate protection and retry stay outside this contract.

INVARIANT

Only completed well-formed text succeeds

A truncated, incomplete, empty or malformed response never yields partial text.

mechanism

INVARIANT

Preparation performs no I/O

Audio admission and request encoding allocate and validate locally; headers stay behind an inspection-safe closure.

mechanism

PRINCIPLE

Leave transport authority with the caller

The supplied HTTP client keeps its limits, deadline, cancellation, trust and destinations. The operation adds no retry and no client lifetime.

4.2 Transcription

One inline audio request returns completed text while the caller keeps transport and recovery policy.

Inline transcription

responsibility

Admit inline bytes, encode one Google Interactions request and return completed nonempty text.

interface

transcribe.audio validates bytes and MIME under the caller's raw byte allowance; settings(model) defaults to automatic language detection and Verbatim. google(key) creates Config, with_endpoint and with_request_limit configure it, prepare returns opaque Prepared, request_json exposes only its body, and run(client, prepared) returns String or the common Failure.

interactions

The application supplies model, language hints, mode and its existing HTTP Gun client. It may place the returned String in its own record or a later generation request.

invariants

Preparation performs no I/O. Encoded headers remain behind a closure. Execution neither retries nor changes the client's policy or lifetime.

failure

AudioError distinguishes invalid bytes, unsupported MIME, invalid allowance and excess raw bytes. Preparation returns PrepareError. Executed failure retains provider and conservative send evidence.

Inline audio

value-object · immutable · transcription · owned by transcription admission

Nonempty byte-aligned audio declared by the caller.

ATTRIBUTES

authored **Bytes and MIME** · BitArray and admitted MIME String
The constructor bounds raw bytes before Base64 encoding. It admits the provider's documented WAV, MP3, AIFF, AAC, OGG, FLAC, MPEG, M4A, L16, Opus, ALAW, MULAW and WebM MIME strings. It does not inspect codecs, duration or acoustic validity.

authored **Allowance** · positive byte count
The caller owns allocation of the original input. Admission cannot retroactively bound that allocation.

Prepared transcription

value-object · immutable · transcription · owned by transcription admission

One admitted encoded inline request, reusable only by explicit caller choice.

ATTRIBUTES

derived **Input and settings** · encoded request body and endpoint
Settings keeps a required nonempty model, language-code strings and Verbatim or Smart. Empty language hints mean provider autodetection; provider/model compatibility is checked remotely. The request sets store to false.

derived **Credentials** · inspection-safe header closure
Preparation validates and captures a nonempty visible-ASCII API key. Changing credentials requires preparing again. request_json contains audio data and is sensitive even though it excludes credentials.

RELATIONSHIPS

1 : 0..n A prepared value can be executed repeatedly. It is not an idempotency receipt or a durable execution record.

- The default endpoint is the Google HTTPS Interactions endpoint. Overrides are complete HTTPS URLs with a nonempty host/path and no userinfo, query, fragment or controls. Client destination/TLS policy still applies. Request JSON is limited after encoding, by default to 16 MiB; with_request_limit changes that positive allowance. Raw admission, encoded admission and HTTP response bounds protect different allocations.
- run borrows the client unchanged: response byte/overflow bounds, absolute deadline, timeout, cancellation, trust and destinations remain the caller's. A truncated buffered result is always refused. This family adds no first-token timer, SSE owner or universal memory guarantee. JSON depth is checked before decoding; synchronous encoding/decoding has no independent preemption.
- Only a completed response with well-formed model-output text produces a nonempty trimmed String. Text fragments retain order; reasoning and other typed content are excluded. Malformed text refuses the whole result; incomplete, empty, invalid UTF-8 or invalid JSON are protocol failures.
- A non-success response retains status and Retry-After with no response body. A transport failure preserves the HTTP failure and NotSent or MaybeSent; a complete unusable response has Completed evidence. Truncation has MaybeSent evidence and never yields partial text. Failure.partial_output is false because this operation emits no progress.
- Started, buffered RequestSent, Terminal and Cleanup observations reuse the package event and caller correlation. They contain no audio, key or transcript. Cleanup describes local transport, not remote cancellation.
- File upload, remote resource management, diarization, timestamps, custom vocabulary, streaming audio, persistence, duplicate protection and retry are outside this contract. These remain caller or explicitly deferred capabilities. ADR-0009 explains the separate family.

BEHAVIOR

Invalid audio or configuration is refused before execution

Given Audio is empty, misaligned, oversized or has an unsupported MIME, or its preparation has invalid credentials, request settings or encoded allowance.

When The caller constructs the audio or prepares it.

Then A typed refusal identifies the input boundary and no execution starts.

level: interface area: Transcription admission

BEHAVIOR

Only completed usable transcription becomes text

Given A prepared inline transcription receives a completed provider response.

When The response is interpreted.

Then Well-formed model text returns as one ordered trimmed nonempty String; unusable or incomplete evidence returns a typed failure.

level: interface area: Transcription result

BEHAVIOR

Transcription preserves caller transport authority

Given The caller executes a prepared transcription through its configured transport view.

When The request succeeds, fails, is cancelled, exceeds a bound or returns truncated data.

Then One attempt preserves that view's restrictions, closes its response and leaves the shared client usable.

Then Truncated data is refused and no automatic retry or remote rollback is claimed.

level: boundary area: Transcription execution

Glossary

Every term the layer declares, with the context that owns it.

Answer candidate

owned by classification

An untrusted provider-neutral yes probability, selected choice or rated rubric emitted by a classification wire. Shared admission determines whether it matches the authored questions.

Application deadline

owned by generation

An explicit absolute deadline carried by the HTTP client the caller supplies. A generation can shorten its own budget below it and cannot extend it.

Assistant turn

owned by generation

One response's text, application tool calls, provider identity and message-local replay data. It contains no conversation history or execution capability.

Classification config

owned by classification

Live settings holding fresh credentials, an optional endpoint override, a timeout and live byte allowances. It selects no wire.

Classification receipt

owned by classification

The requested model, state and original request and response evidence from which a native classification outcome is reconstructed. Its codec captures a pure wire and questions rather than live credentials.

Classification wire

owned by classification

A pure provider protocol projection with its identity, default endpoint and fixed receipt evidence bounds. Its lifetime is independent of live execution settings.

Concentration evidence

owned by classification

An optional provider-reported confidence measurement of the answer distribution. It is not the probability that the answer is correct.

Generation settings

owned by generation

An immutable value holding optional maximum output tokens, temperature, top-p, stop sequences and one provider cache reference. It holds no model, messages, tools, credentials or output codec.

Inline audio

owned by transcription

Nonempty byte-aligned audio with an admitted MIME string, bounded by the caller's raw byte allowance before Base64 encoding. Admission does not inspect codec, duration or acoustic validity.

Live recording

owned by generation

An opt-in run that sends real requests to a commercial provider and publishes a sanitized cassette. It spends provider tokens and is never part of an ordinary gate.

Observation

owned by generation

A bounded lifecycle fact reported through Sinal with call identity, correlation, stage, provider and outcome. It carries no content or credentials and authorizes nothing.

Oracle evidence

owned by generation

Selected behavior of a pinned upstream implementation compared against local assertions. It establishes named laws and no parity with the upstream suite.

Prepared generation

owned by generation

One admitted and encoded generation request with its provider, settings and output decoder. It is a reusable value rather than an executing operation.

Prepared transcription

owned by transcription

One admitted encoded inline request with its endpoint and inspection-safe headers. It is reusable by explicit caller choice and is neither an idempotency receipt nor a durable record.

Provider adapter

owned by generation

The encoder, schema projection, header callbacks and event reducer that give one provider protocol its meaning. Shared admission, limits, timers and cleanup stay outside it.

Provider delay

owned by generation

The provider's readable Retry-After duration, distinguished from caller-selected backoff. It has no package-imposed maximum.

Public consumer

owned by generation

A separate package that depends on this library through public imports only, exercises its adoption path, including compile-time refusals, and holds the compiled, offline programs that serve as the usage documentation.

Question batch

owned by classification

Named classification questions retaining application-native values and a decoder for their combined answer. Heterogeneous batches combine their native answers as tuples.

Receipt bounds

owned by classification

The fixed request and response byte allowances of a wire that govern stored evidence. Live byte allowances cannot exceed them.

Replay data

owned by generation

Provider-owned response data required to encode that assistant turn in a later caller-supplied conversation. It is not a durable execution checkpoint.

Retry advice

owned by generation

A prospect for another attempt and the owner of the wait, derived from one failure alone. It schedules nothing and does not say that a repeat is safe.

Semantic progress

owned by generation

Accepted text, refusal, reasoning or tool-argument output from a generation response. Usage reports and extension notifications are not semantic progress.

Semantic test reply

owned by generation

An opaque description of a provider response, such as text, tool calls, refusal or failure, lowered through the real provider reducer into scripted HTTP exchanges.

Send evidence

owned by generation

The conservative classification NotSent, MaybeSent or Completed attached to a failed interaction. It describes observed reachability and completion rather than effect rollback or replay safety.

Stream owner

owned by generation

The authoritative process holding one generation execution's framing, reduction, delivery and terminal state. Copied stream handles name the same owner.

Tool call issue

owned by generation

An undeclared tool name or invalid argument value reported beside a complete tool-request turn under the reporting policy. It does not establish permission to execute that call.

Transcription mode

owned by transcription

The typed choice between Verbatim and Smart output for one request. Language support remains a provider decision.